

Enterprise AI Guard: Complete Project Report

1. Executive Summary

The **Enterprise AI Guard** is a comprehensive, zero-trust Data Leak Prevention (DLP) platform designed to protect corporate assets from being accidentally or maliciously leaked to public Generative AI platforms like ChatGPT, Claude, and Gemini.

Rather than relying on network-level proxies which break end-to-end encryption, this project utilizes a client-side Chrome Extension paired with a centralized FastAPI backend to analyze, sanitize, and block sensitive prompts in real-time, right inside the employee's browser.

2. Technical Stack

- **Backend:** Python 3.10+, FastAPI, Uvicorn
- **Database:** Embedded SQLite (safety_audit.db) for serverless, zero-config data persistence.
- **Detection Engine:** Native Python Regex, Microsoft Presidio Analyzer (spaCy NLP).
- **Browser Extension:** Chrome Manifest V3, Vanilla JavaScript, CSS.
- **Web Dashboard:** HTML5, Tailwind CSS, Chart.js for analytics.

3. Architectural Components

A. The Browser Extension (The Client Shield)

The extension acts as the primary enforcement layer on the employee's machine.

- **Real-Time Interception:** A debounced content script (content.js) listens to keystrokes inside AI chat windows.
- **Safe React Intercepts:** Instead of hijacking standard submit buttons, the extension gracefully intercepts the Enter key.
- **CSP Bypass:** To bypass strict Content Security Policies on platforms like ChatGPT, the extension uses a Service Worker to securely route fetch requests to the local backend.
- **Local UI:** Features a sleek, Vercel-inspired dark mode Popup and Options page.

B. The Detection Engine (The Brain)

The Python-based SafetyScanner evaluates text through three distinct security phases:

1. **Custom Proximity Matching:** Uses Anchor-Value logic. It searches for custom keywords defined by the CISO (e.g., "password", "api").
2. **Static Signature Rules:** Uses strict mathematical RegEx patterns to instantly identify standard secrets (AWS Keys, RSA Private Keys, Credit Cards).
3. **Semantic NLP (Microsoft Presidio):** Uses Artificial Intelligence to read the context of the sentence, identifying Personally Identifiable Information (PII).

C. The Central Backend & Database (The Command Center)

The FastAPI backend serves as the central hub for the enterprise.

- **Stateless API:** Exposes endpoints for prompt scanning, rule management, and keyword management.
- **Immutable Auditing:** Every scan is logged to the SQLite database.

D. The CISO Dashboard (The UI)

A centralized web dashboard designed for security administrators.

- **Analytics:** Visualizes risk category distributions using Chart.js doughnut charts.

- **Live Audit Trail:** A real-time table displaying every prompt intercepted.

- **Global Policy Management:** Allows administrators to dynamically add new Custom Monitored Keywords.

4. Risk Scoring & Remediation

Every violation detected by the engine is assigned a **Threat Weight**. The engine aggregates these weights to calculate a final Risk Score:

- **ALLOW (Score 0-24):** The prompt is safe. The extension forwards it to the AI natively.
- **WARN (Score 25-39):** Minor PII detected. The extension halts the prompt and shows a floating warning UI, allowing the user to override.
- **BLOCK (Score 40+):** Critical data detected (e.g., API Keys, Passwords). The extension entirely blocks the transmission.

5. Security & Privacy

- **Zero External Dependencies:** Because the platform relies entirely on an embedded SQLite database and a local Python server, no corporate data is ever transmitted to a third-party cloud provider for analysis.
- **Sanitization:** When a prompt is flagged, the backend automatically generates a sanitized version of the text (masking the sensitive data).